

Министерство науки и высшего образования РФ
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра Информационные технологии и автоматизированные системы

Базы данных
Лабораторная работа № 3

Тема: «Вывод результатов SQL-запросов на страницу браузера»

Выполнил: студент

группы _____ РИС-25-26

_____ Ильинов Ф. М.

Проверил: доцент кафедры ИТАС

Петренко А.А.

г. Пермь – 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Задачи работы	4
3	Этапы выполнения.....	5
3.1	Проектирование и разработка веб-сервера	6
3.1.1	Точка входа в программу	6
3.1.2	Обработчик главной страницы.....	9
3.1.3	Обработчик SQL-запросов.....	9
3.2	Проектирование и разработка веб-интерфейса	14
3.3	Результат выполнения программы	18
	Заключение	21
	Список использованных источников	22

1 Цель работы

Реализовать вывод трёх SQL запросов на страницу браузера:

- а) Получить список сотрудников, номера телефонов и ЗП (листинг 1);
- б) Получить список сотрудников с их адресами (отсортировать по адресу) (листинг 2);
- в) Получить список сотрудников с продолжительностью трудовой деятельности больше 4 лет (листинг 3).

```
1 SELECT last_name, first_name, middle_name, phone_number, salary
2 FROM lab2_employee_ilinov
3 ORDER BY last_name, middle_name, first_name;
```

Листинг 1 – SQL запрос для получения списка сотрудников, их номеров телефонов и ЗП

```
1 SELECT last_name, first_name, middle_name, address
2 FROM lab2_employee_ilinov
3 ORDER BY address, last_name, middle_name, first_name;
```

Листинг 2 – SQL запрос для получения списка сотрудников с их адресами

```
1 SELECT last_name, first_name, middle_name, date_start
2 FROM lab2_employee_ilinov
3 WHERE extract(days from now() - date_start) / 365 > 4
4 ORDER BY last_name, middle_name, first_name;
```

Листинг 3 – SQL запрос для получения списка сотрудников, чья трудовая деятельность больше 4 лет

2 Задачи работы

В соответствии с целью были поставлены следующие задачи:

- а) Спроектировать и разработать локальный веб-сервер;
- б) Спроектировать и разработать веб-интерфейс;
- в) Вывести результаты запросов на страницу браузера.

3 Этапы выполнения

Для выполнения поставленных задач были сформулированы следующие технологические требования к инструментарию:

- Поддержка работы с СУБД PostgreSQL [1];
- Поддержка работы с протоколом HTTP [2];
- Возможность отображения кнопок в веб-интерфейсе;
- Возможность отображения таблиц в веб-интерфейсе.

Для работы с веб-страницами был выбран язык разметки HTML [3], так как он позволяет создавать кнопки, требуемые для выбора текущего SQL запроса, и отрисовывать таблицы.

В качестве языка программирования был выбран Golang [4], так его стандартные библиотеки реализовывают широкий и удобный функционал для работы с веб-серверами.

Программа реализует принцип Server-Side Render. Итоговые HTML собираются на стороне сервера и отправляются клиенту.

3.1 Проектирование и разработка веб-сервера

При разработке сервера были использованы следующие библиотеки (листинг 4)

```
3 import (  
4     "html/template"  
5     "log/slog"  
6     "net/http"  
7     "os"  
8  
9     "github.com/jackc/pgx"  
10 )
```

Листинг 4 — Используемые библиотеки

- а) `html/template` — используется для формирования итоговых HTML;
- б) `log/slog` — используется для ведения журнала ошибок;
- в) `net/http` — реализует работу с протоколом `http`;
- г) `os` — требуется для завершения программы с кодом 1 в случае возникновения критической ошибки;
- д) `github.com/jackc/pgx` — используется для работы с СУБД PostgreSQL.

3.1.1 Точка входа в программу

Функция `main` является точкой входа в программу в языке Golang (листинг 5).

```

93 func main() {
94     cssFS := http.FileServer(http.Dir("../css"))
95     htmlTemplate, err := template.ParseFiles("index.html")
96     if err != nil {
97         slog.Error("cannot parse index.html", slog.Any("err", err))
98         os.Exit(1)
99     }
100
101     pool, err := pgx.NewConnPool(pgx.ConnPoolConfig{
102         Host:      "localhost",
103         Port:      5432,
104         Database: "postgres",
105         User:      "postgres",
106         Password: "password",
107     })
108
109     if err != nil {
110         slog.Error("cannot connect to postgres", slog.Any("err",
111             ↪ err))
112         os.Exit(1)
113     }
114
115     mux := http.NewServeMux()
116     mux.Handle("GET /{$}", Root(htmlTemplate))
117     mux.Handle("GET /css/", http.StripPrefix("/css/", cssFS))
118     mux.Handle("GET /query", Queries(htmlTemplate, pool))
119
120     slog.Info("starting server", slog.Any("address", httpAddress))
121
122     if err = http.ListenAndServe(httpAddress, mux); err != nil {
123         slog.Error("cannot listen server", slog.Any("err", err),
124             ↪ slog.Any("address", httpAddress))
125         os.Exit(1)
126     }
127 }

```

Листинг 5 — Точка входа в программу — функция main

В строке 94 инициализируется обработчик HTTP-файлового сервера, требуемого для раздачи файла стилей css. Этот файл определяет только визуальное оформление сайта. В его качестве была выбрана открытая библиотека стилей `pico.css` [5]

Обработчик — функция, принимающая типы `http.ResponseWriter` (используется для записи в HTTP-соединение) и `http.Request` (запрос клиента).

В следующей строке создается шаблон на основе файла `index.html` и проверяется корректность выполнения этой функции. В случае, если функция завершилось ошибкой, выводится соответствующее сообщение в консоль и программа завершает выполнение с кодом 1.

В строках 101–107 инициализируется пул соединений к СУБД, где указывается название хоста, порт подключения, имя базы данных и пользователя, а так же пароль. В строках 109–112 также проверяется наличие ошибки, по аналогии с предыдущим абзацем.

В строках 114–117 создается маршрутизатор (мультиплексор) HTTP-запросов:

а) По пути `/` — главная страница. Для обработки запросов используется функция `Root`, принимающая HTML-шаблон и возвращающая требуемый обработчик. Ключевым словом `GET` определяется допустимый HTTP-метод, а при помощи символов `{ $ }` мы говорим программе, что все иные пути запрещены. В таком случае при попытке зайти на иной путь сервер вернет ошибку 404, а для всех остальных методов помимо `GET` клиенту возвращается HTTP-статус 405 `Method Not Allowed`.

б) По пути `/css/` должны находиться файлы стилей. Используется обработчик, созданный в строке 95.

в) По пути `/query` — результаты выполнения SQL запросов. Функция `queries` принимает HTML-шаблон и созданный в строках 101–107 пул соединений.

В строках 119–124 запускается HTTP-сервер, находящийся по адресу `httpAddress`. Этот адрес задан как константа в начале программы и равен `localhost:8080`

3.1.2 Обработчик главной страницы

Функция `Root` (листинг 6) принимает на вход HTML-шаблон и возвращает HTTP-обработчик.

```
19 func Root(htmlTemplate *template.Template) http.HandlerFunc {  
20     return func(writer http.ResponseWriter, request *http.Request) {  
21         err := htmlTemplate.Execute(writer, nil)  
22         if err != nil {  
23             slog.Error("cannot execute template",  
24                 ↪ slog.Any("err", err))  
25             writer.WriteHeader(  
26                 http.StatusInternalServerError)  
27         }  
28     }  
29 }
```

Листинг 6 — Обработчик главной страницы

Функция собирает итоговый HTML-документ. В качестве параметров сборщика `htmlTemplate.Execute` передаются `writer` и `nil`, так как в данном случае наша страница не содержит никаких данных SQL запросов. В случае ошибки клиенту отправляется статус `500 Internal Server Error`, сигнализирующий об ошибке на стороне сервера.

3.1.3 Обработчик SQL-запросов

Функция `Queries` (листинги 7, 8) работает аналогично функции `Root`, однако помимо шаблона она также принимает объект типа `pgxConnPool`, служащий для выполнения SQL запросов.

```

30 func Queries(htmlTemplate *template.Template, pool *pgx.ConnPool)
    ↪ http.HandlerFunc {
31     return func(writer http.ResponseWriter, request *http.Request) {
32         opId := request.URL.Query().Get("opId")
33         resp := new(Response)
34         var err error
35         switch opId {
36             case "1":
37                 resp, err = execQuery(
38                     pool,
39                     "SELECT last_name, first_name, middle_name,
    ↪ phone_number, salary FROM
    ↪ lab2_employee_ilinov ORDER BY
    ↪ last_name, middle_name, first_name;")
40             case "2":
41                 resp, err = execQuery(
42                     pool,
43                     "SELECT last_name, first_name, middle_name,
    ↪ address FROM lab2_employee_ilinov
    ↪ ORDER BY address, last_name,
    ↪ middle_name, first_name;")
44             case "3":
45                 resp, err = execQuery(
46                     pool,
47                     "SELECT last_name, first_name, middle_name,
    ↪ date_start FROM lab2_employee_ilinov
    ↪ WHERE extract(days from now() -
    ↪ date_start) / 365 > 4 ORDER BY
    ↪ last_name, middle_name, first_name;")
48             default:
49                 writer.WriteHeader(http.StatusBadRequest)

```

Листинг 7 – Обработчик SQL-Запросов (часть 1)

```

49         writer.WriteHeader(http.StatusBadRequest)
50         return
51     }
52     if err != nil {
53         slog.Error("cannot get query", slog.Any("error",
54             ↪ err), slog.Any("queryId", opId))
55         writer.WriteHeader(
56             http.StatusInternalServerError)
57         return
58     }
59     resp.Type = opId
60
61     if err := htmlTemplate.Execute(writer, resp); err != nil {
62         slog.Error("cannot execute template",
63             ↪ slog.Any("err", err))
64         writer.WriteHeader(
65             http.StatusInternalServerError)
66         return
67     }
68 }
69 }
70 }

```

Листинг 8 – Обработчик SQL-Запросов (часть 2)

Далее в ней выполняются следующие шаги:

- а) Получение идентификатора операции `opId` из Query HTTP-запроса;
- б) Формирование объекта структуры `Response` и переменной для ошибок `err`;
- в) Выполнение функции `execQuery`, перезаписывающей переменные `resp` и `err` и принимающей пул запросов и сам SQL-запрос;
- г) Обработка ошибки, возникшей при выполнении функции `execQuery`;
- д) Запись идентификатора операции в параметр `Type` переменной `resp`;
- е) Сборка и отправка итогового HTML-документа, обработка ошибки.

Структура HTTP-ответа (листинг 9) содержит в себе следующие поля:

- `Type` типа `string`. Хранит в себе идентификатор операции;

— Rows типа any. Хранит в себе сами SQL-ответы. Тип any обусловлен тем, что все три запроса имеют разную структуру ответа.

```
14 type Response struct {
15     Type string
16     Rows any
17 }
```

Листинг 9 — Структура, используемая для HTTP-ответа

Функция `execQuery` (листинг 10) используется для выполнения SQL запросов.

```
69 func execQuery(pool *pgx.ConnPool, query string) (*Response, error) {
70     rows, err := pool.Query(query)
71     if err != nil {
72         return nil, err
73     }
74     defer rows.Close()
75
76     result := make([]any, 0)
77     for rows.Next() {
78         row, err := rows.Values()
79         if err != nil {
80             return nil, err
81         }
82         result = append(result, row)
83     }
84     if err := rows.Err(); err != nil {
85         return nil, err
86     }
87
88     resp := new(Response)
89     resp.Rows = result
90     return resp, nil
91 }
```

Листинг 10 — Вспомогательная функция `execQuery`

В качестве аргументов она принимает пул соединений СУБД и сам запрос. Возвращает объект структуры `Response` и сообщение об ошибке. В первых строках выполняется сам SQL запрос и обрабатывается ошибка. В случае её возникновения функция завершается и возвращает полученную ошибку.

Далее создается массив `rows`, хранящий строки ответа. В цикле `for` при помощи метода `exes.Next()` проверяется, остались ли ещё в ответе необработанные строки. Каждая строка кодируется в массив значений при помощи `exes.Values()`, обрабатывается ошибка, и в случае успеха значения записываются в массив `rows`.

Далее SQL-запрос завершается, формируется ответ, и происходит возврат из функции.

3.2 Проектирование и разработка веб-интерфейса

Как уже было сказано ранее, сервис реализует принцип Server-Side Render. Для работы с полученными сервером данными используются специальные операнды и операции, заключенные в двойные фигурные скобки.

В начале документа указываются (листинг 11):

- а) Тип документа — HTML;
- б) Язык документа — русский;
- в) Технические параметры для браузера:
 - 1) Кодировка UTF-8 для поддержки кириллицы;
 - 2) Заголовок страницы;
 - 3) Подключается файл стилей pico.classless.min.css.

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <title>БД Лабораторная работа №3</title>
6   <link rel="stylesheet" href="/css/pico.classless.min.css">
7 </head>
```

Листинг 11 — Начало HTML-документа

В начале тела контейнера main указываются заголовки страницы (листинг 12):

- а) Дисциплина и номер работы;
- б) Группа и ФИО студента.

```
9 <main class="container">
10   <header>
11     <h1>БД Лабораторная работа №3</h1>
12     <p>РИС-25-26 Ильинов Ф. М.</p>
13   </header>
```

Листинг 12 — Заголовки страницы

В первом блоке `article` создаются кнопки, используемые для вызова SQL-запросов (листинг 13):

а) Строки `<form action="/query" method="get">` определяют путь запроса и его метод;

б) Строки `<input type="hidden" name="opId" value="<идентификатор_операции>">` создают query-параметр `opId` и присваивают ему значение равное идентификатору операции;

в) Объекты `<button type="submit">` создают кнопки и надписи на них.

```
15 <article>
16   <form action="/query" method="get">
17     <input type="hidden" name="opId" value="1">
18     <button type="submit">Получить список сотрудников,
19       номера телефонов и ЗП</button>
20   </form>
21   <form action="/query" method="get">
22     <input type="hidden" name="opId" value="2">
23     <button type="submit">Получить список сотрудников
24       с их адресами (отсортировать по адресу)</button>
25   </form>
26   <form action="/query" method="get">
27     <input type="hidden" name="opId" value="3">
28     <button type="submit">Получить список сотрудников
29       с продолжительностью трудовой деятельности больше 4 лет
30     </button>
31   </form>
32 </article>
```

Листинг 13 – Первый блок `article`

Во втором блоке `article` реализована логика формирования таблицы SQL-запросов (листинг 14).

```

34 <article>
35   <table>
36     <thead>
37       {{ if eq .Type "1" }}
38         <th>Фамилия</th>
39         <th>Имя</th>
40         <th>Отчество</th>
41         <th>Номер телефона</th>
42         <th>Зарплата</th>
43       {{ else }} {{ if eq .Type "2" }}
44         <th>Фамилия</th>
45         <th>Имя</th>
46         <th>Отчество</th>
47         <th>Адрес</th>
48       {{ else }} {{ if eq .Type "3" }}
49         <th>Фамилия</th>
50         <th>Имя</th>
51         <th>Отчество</th>
52         <th>Дата найма</th>
53       {{ else }}
54         Нажмите кнопку
55       {{ end }}
56     {{ end }}
57   {{ end }}
58   </thead>
59   {{ range .Rows }}
60   <tr>
61     {{ range . }}
62     <td>{{ . }}</td>
63     {{ end }}
64   </tr>
65   {{ end }}
66 </table>
67 </article>

```

Листинг 14 — Логика формирования таблицы

Используются элементы `<table>`, `<thead>` для инициализации таблицы и заголовков её столбцов.

Далее в двойных фигурных скобках проверяется условие: если поле `Type` структуры, полученной в результате выполнения, равно единице, то при помощи элементов `<th>` создаются заголовки: фамилия, имя, отчество, номер телефона, зарплата.

В ином случае, если тип равен двум: фамилия, имя, отчество, адрес.

В ином случае, если тип равен трем: фамилия, имя, отчество, дата най-
ма.

В ином случае вместо заголовков пишется строка «Нажмите кнопку».

Далее срабатывает цикл `{{ range .Rows }}`, который используется для перебора всех элементов массива `Rows`. Для каждого элемента создается блок `<tr>`, инициализирующий строку таблицы. Внутри каждого блока при помощи цикла `{{ range . }}` перебираются все элементы строки и при помощи `<td>` записываются по очереди в каждую ячейку строки.

3.3 Результат выполнения программы

Скриншот главной страницы приложения (рис. 3.1)

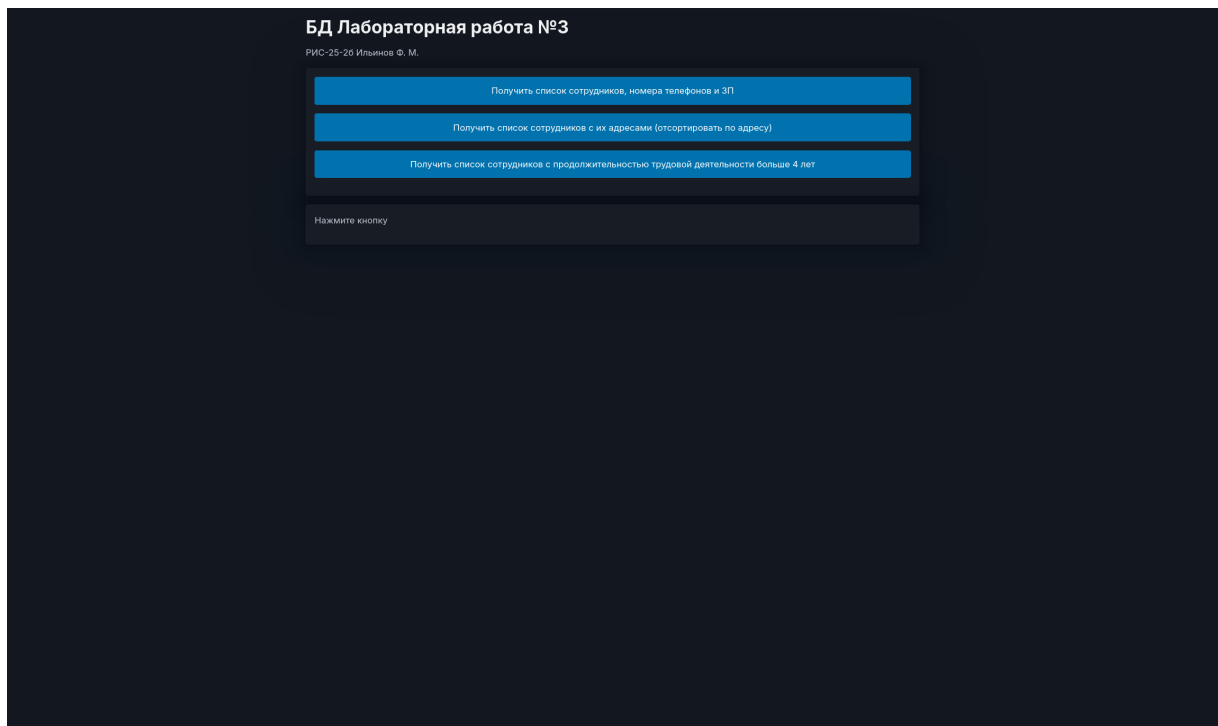


Рисунок 3.1 — Главная страница

Скриншот выполнения первого запроса: список сотрудников с номерами телефонов и заработной платой (рис. 3.2)

БД Лабораторная работа №3

РИС-25-26 Ильинов Ф. М.

Получить список сотрудников, номера телефонов и ЗП

Получить список сотрудников с их адресами (отсортировать по адресу)

Получить список сотрудников с продолжительностью трудовой деятельности больше 4 лет

Фамилия	Имя	Отчество	Номер телефона	Зарплата
Богданов	Михаил	Александрович	+7 (950) 444-55-66	130000
Васильев	Артём	Викторович	+7 (905) 999-88-77	98000
Виноградова	Юлия	Валерьевна	+7 (961) 888-77-66	93000
Волков	Илья	Олегович	+7 (927) 444-33-22	105000
Воробьева	Светлана	Геннадьевна	+7 (901) 333-22-11	67000
Голубев	Владимир	Максимович	+7 (992) 111-00-99	86000
Зайцева	Екатерина	Романовна	+7 (919) 333-44-55	79000
ИльиИО	Федос	Микаэлович	+7 (913) 555-12-34	95000
Ильинов	Фёдор	Михайлович	+7 (916) 123-45-67	75000
ИльиИИФФ	Федя	Михалыч	+7 (921) 987-65-43	82000
Козлова	Анна	Дмитриевна	+7 (937) 666-55-44	72000
Кузнецова	Елена	Васильевна	+7 (922) 444-55-66	68000
Лебедев	Сергей	Михайлович	+7 (951) 222-33-44	125000
Морозова	Ирина	Юрьевна	+7 (918) 888-99-00	85000
Новиков	Максим	Андреевич	+7 (904) 111-22-33	90000
Павлов	Павел	Денисович	+7 (902) 777-66-55	115000
Попов	Алексей	Николаевич	+7 (987) 333-22-11	110000
Семенова	Татьяна	Евгеньевна	+7 (960) 222-11-00	71000
Соколова	Ольга	Владимировна	+7 (903) 777-88-99	88000
Соловьева	Наталья	Анатольевна	+7 (908) 555-66-77	64000

Рисунок 3.2 – Результаты выполнения первого запроса

Скриншот выполнения второго запроса: список сотрудников, отсорти-
рованный по адресу (рис. 3.3)

БД Лабораторная работа №3

РИС-25-26 Ильинов Ф. М.

Получить список сотрудников, номера телефонов и ЗП

Получить список сотрудников с их адресами (отсортировать по адресу)

Получить список сотрудников с продолжительностью трудовой деятельности больше 4 лет

Фамилия	Имя	Отчество	Адрес
Виноградова	Юлия	Валерьевна	г. Барнаул, ул. Попова, д. 3, кв. 81
Павлов	Павел	Денисович	г. Волгоград, пр. Университетский, д. 7, кв. 98
Васильев	Артём	Викторович	г. Воронеж, ул. Плехановская, д. 19, кв. 60
Кузнецова	Елена	Васильевна	г. Екатеринбург, ул. Мира, д. 23, кв. 5
Богданов	Михаил	Александрович	г. Ижевск, ул. Пушкинская, д. 22, кв. 5
Воробьева	Светлана	Геннадьевна	г. Иркутск, ул. Байкальская, д. 11, кв. 29
Попов	Алексей	Николаевич	г. Казань, ул. Баумана, д. 12, кв. 89
Соловьева	Наталья	Анатольевна	г. Красноярск, ул. Карла Маркса, д. 4, кв. 11
Ильинов	Фёдор	Михайлович	г. Москва, ул. Ленина, д. 10, кв. 15
Соколова	Ольга	Владимировна	г. Нижний Новгород, ул. Горького, д. 100, кв. 3
ИльиИО	Федос	Микаэлович	г. Новосибирск, ул. Кирова, д. 8, кв. 44
Новиков	Максим	Андреевич	г. Омск, ул. Гагарина, д. 5, кв. 17
Зайцева	Екатерина	Романовна	г. Пермь, ул. Ленина, д. 52, кв. 34
Морозова	Ирина	Юрьевна	г. Ростов-на-Дону, ул. Садовая, д. 88, кв. 9
Козлова	Анна	Дмитриевна	г. Самара, ул. Советская, д. 14, кв. 22
ИльиИИФФ	Федя	Михалыч	г. Санкт-Петербург, Невский пр., д. 45, кв. 12
Семенова	Татьяна	Евгеньевна	г. Саратов, ул. Чалаева, д. 15, кв. 40
Голубев	Владимир	Максимович	г. Тюмень, ул. Республики, д. 90, кв. 14
Волков	Илья	Олегович	г. Уфа, ул. Октября, д. 31, кв. 76
Лебедев	Сергей	Михайлович	г. Челябинск, пр. Ленина, д. 67, кв. 51

Рисунок 3.3 – Результаты выполнения второго запроса

Скриншот выполнения третьего запроса: список сотрудников с продолжительностью трудовой деятельности больше 4 лет (рис. 3.4)

БД Лабораторная работа №3
РИС-25-26 Ильинов Ф. М.

Получить список сотрудников, номера телефонов и ЗП

Получить список сотрудников с их адресами (отсортировать по адресу)

Получить список сотрудников с продолжительностью трудовой деятельности больше 4 лет

Фамилия	Имя	Отчество	Дата найма
Богданов	Михаил	Александрович	2018-10-01 00:00:00 +0000 UTC
Васильев	Артём	Викторович	2020-03-02 00:00:00 +0000 UTC
Виноградова	Юлия	Валерьевна	2019-09-23 00:00:00 +0000 UTC
Волков	Илья	Олегович	2019-05-27 00:00:00 +0000 UTC
Воробьева	Светлана	Геннадьевна	2022-05-18 00:00:00 +0000 UTC
Голубев	Владимир	Максимович	2020-01-15 00:00:00 +0000 UTC
Зайцева	Екатерина	Романовна	2021-12-01 00:00:00 +0000 UTC
ИльинЮ	Федос	Микаэлович	2019-11-10 00:00:00 +0000 UTC
Ильинов	Фёдор	Михайлович	2021-03-15 00:00:00 +0000 UTC
Ильинофф	Федя	Михалыч	2020-06-01 00:00:00 +0000 UTC
Кузнецова	Елена	Васильевна	2022-01-20 00:00:00 +0000 UTC
Лебедев	Сергей	Михайлович	2017-02-15 00:00:00 +0000 UTC
Морозова	Ирина	Юрьевна	2021-07-19 00:00:00 +0000 UTC
Новиков	Максим	Андреевич	2020-10-05 00:00:00 +0000 UTC
Павлов	Павел	Денисович	2018-08-20 00:00:00 +0000 UTC
Попов	Алексей	Николаевич	2018-04-12 00:00:00 +0000 UTC
Соколова	Ольга	Владимировна	2021-09-01 00:00:00 +0000 UTC

Рисунок 3.4 – Результаты выполнения третьего запроса

ЗАКЛЮЧЕНИЕ

В процессе выполнения работы были освоены навыки работы с языком разметки HTML, работа с СУБД PostgreSQL в языке Golang, работа с протоколом HTTP в языке Golang.

Также был реализован вывод результатов трех SQL запросов на страницу браузера:

- а) Получить список сотрудников, номера телефонов и ЗП (листинг 1);
- б) Получить список сотрудников с их адресами (отсортировать по адресу) (листинг 2);
- в) Получить список сотрудников с продолжительностью трудовой деятельности больше 4 лет (листинг 3).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Documentation PostgreSQL 18.6. The PostgreSQL Global Development Group. — 2026. — Режим доступа: <https://www.postgresql.org/files/documentation/pdf/18/postgresql-18-A4.pdf> (дата обращения: 01.09.2026).
2. Wikipedia. HTTP. — 2026. — Режим доступа: <https://en.wikipedia.org/wiki/HTTP> (дата обращения: 01.09.2026).
3. Mozilla. HTML: HyperText Markup Language. — 2026. — Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/HTML> (дата обращения: 01.09.2026).
4. Google. The Go Programming Language. — 2026. — Режим доступа: <https://go.dev/> (дата обращения: 01.09.2026).
5. Larroche Lucas. Pico CSS • Minimal CSS Framework for semantic HTML. — 2026. — Режим доступа: <https://picocss.com/> (дата обращения: 01.09.2026).